# Clinch: Sybil-proof token launches and batch swaps on Uniswap v4

Team Clinch · ETHGlobal Mumbai, November 2026 · Version 2.0

github.com/asengupta07/clinch · full version at /whitepaper.pdf on the Clinch app

**Abstract.** Onchain identities are free, so any sale that looks at wallets (caps, pro rata, lotteries) can be beaten by one person with many wallets. Zhang et al. [1] proved in October 2026 that adaptive clinching, the budget-constrained auction of Dobzinski, Lavi and Nisan [2], keeps truthful bidding dominant even when a bidder splits across any number of identities, as long as its total budget is enforceable. They also extended it to a two-sided market against a constant-product pool. Clinch implements both on Uniswap v4: a token launch that seeds a pool at the price it discovers, and a hook that clears swaps in short batches. Escrowing every budget makes the theorem's key assumption hold on chain. In our tests no split or misreport gained more than $10^{-5}$ of a bidder's budget at a 0.1% price step, and on Sepolia a 1,000,000-token launch settled in 99 ladder steps for 3.68M gas.

## 1. The problem

Token launches are auctions with budgets, run on chains where a new address costs nothing. A per-wallet cap invites fifty wallets: in our simulation a whale's utility went from 40 with one wallet to 602 with fifty under a capped fixed-price sale. A uniform-price auction (the model behind Uniswap's CCA [7]) does not care about wallet count, but it is not truthful: the same whale raised its utility by 44% by understating its demand. Swaps have a related problem. On a plain pool the order of trades is worth money, so sandwich bots trade before and after a victim [8]. Batch auctions remove the ordering game [9], but the usual batch rules bring back the incentive to misreport.

We wanted three things at once with no identity layer: bidding your true value and budget is best, splitting into wallets never helps, and nobody has to prove they are a person.

## 2. The result we implement

In an adaptive clinching auction the price rises, and at each price a bidder "clinches" the units the other bidders can no longer afford, paying the current price [2, 3]. It was already known to be truthful for budget-limited bidders [2, 4]. Zhang et al. [1] prove that in this auction, bidding truthfully as one identity weakly dominates every strategy that splits the bidder into finitely many identities with any reported values, provided the identities' combined budget is at most the true budget (their Theorem 1.3). They extend this to an AMM, where the pool's cost curve plays the role of supply, and to two-sided batches (Theorems 1.1 and 6.6). They also show that if budgets can be overreported, no AMM mechanism gets these properties (Theorem 1.2).

On a blockchain that condition is easy to meet. In Clinch a bid only counts after its full budget is locked in the contract, so nobody can report more budget than they have. The one assumption code cannot enforce is that block builders do not censor chosen bids, which [1] and [5] also assume.

## 3. The mechanism

Prices are "payment asset per sale asset". Bid $i$ has a limit $v_i$ and remaining budget $b_i$.

**Demand and clinching.** At clock price $p$ a bid demands $D_i = b_i/p$ while $p < v_i$. Bid $i$ clinches $\max\Big(0, S - \sum_{j\neq i} D_j\Big)$, where $S$ is the residual supply. With $B = \sum b_j$ this is easier as a water level:

$$h = B - pS, \qquad \text{bid } i \text{ buys } \frac{b_i - h}{p} \text{ whenever } b_i > h. \tag{1}$$

Buying $\delta$ at price $p$ lowers both $B$ and $pS$ by $p\delta$, so $h$ is unchanged and the order of purchases does not matter.

**Supply.** A launch sells a fixed $S_0$, so $S = S_0 - X$ after $X$ units are sold. A pool with reserves $(x_0, y_0)$ has cost curve $C(z) = y_0 z/(x_0 - z)$. Its supply at price $p$, given $X$ units sold for $Y$ so far, is the largest $z$ with $C(z) \leq Y + p(z - X)$ [1, eq. 11]. With $K = Y - pX$ that is the larger root of

$$pz^2 + (y_0 + K - px_0)z - Kx_0 = 0, \tag{2}$$

computed without cancellation as $2Kx_0/\big(b + \sqrt{D}\big)$ when $b \geq 0$ and $\big(\sqrt{D} - b\big)/2p$ otherwise, with $b = y_0 + K - px_0$ and $D = b^2 + 4pKx_0$. Buying at price $p$ leaves $K$ unchanged, so the pool behaves like a fixed supply between price steps, and the final payment lands exactly on the curve.

**Two-sided batches.** Buyers and sellers marketable at spot $p_0$ are compared in token0 terms. The smaller side, of size $M$, is filled entirely at spot. The larger side runs the auction against the extended curve $\tilde{C}_M(z) = p_0 z$

for $z \le M$ and $p_0 M + C(z - M)$ beyond it [1, eq. 57]. Only the residual touches the pool, so matched volume never moves the price.

**Ladder and stopping.** The clock moves on a geometric ladder $p_{k+1} = \min(p_k(1+\varepsilon), \text{next bidder limit})$, so every limit lands exactly on a step. The auction stops when active demand fits the supply. Active bids then get their full demand, and bids exiting at that price share the rest in proportion to budget.

**Termination.** With total budget $B_0$ the clock never passes $P_B = C'(C^{-1}(B_0))$ [1, Lemma 6.2], which for a constant-product pool is

$$P_B = \frac{(y_0 + B_0)^2}{x_0 y_0}, \qquad \text{steps} \le \frac{2\ln(1 + B_0/y_0)}{\ln(1+\varepsilon)} + n + 1. \tag{3}$$

A fixed-supply launch has no such bound, so every launch has a price ceiling, and the factory rejects any launch whose worst case would not fit in one block.

**Settlement.** On a ladder, buyers pay the top of each step, which overcharges by about $0.9\varepsilon$. So the hook swaps everything the larger side paid into the pool and shares what actually comes out in proportion to each allocation. Every order gets a small rebate, +0.0028% on a live batch. This departs from the exact settlement in [1]; in our tests it reduced the worst deviation gain.

**Integers.** Prices are $10^{18}$-scaled ratios of raw token units. Amounts round down, payments round up, and every rounding inside Equation 2 pushes $z$ down, so the pool is never underpaid. An early version got this wrong by 1 to 2 wei in 26 of 300 runs; after the fix there were none in 2,000.

## 4. What we built

- **ClinchLaunch.** Bidders escrow a budget and a maximum price until the end time. Anyone can call `settle()`, which runs the ladder and seeds a full-range v4 pool at the clearing price with part of the proceeds and the unsold tokens. The liquidity stays locked. If nobody settles in time, anyone can cancel and everyone is refunded.
- **ClinchHook**, a Uniswap v4 [6] hook (`beforeInitialize`, `beforeAddLiquidity`, `beforeSwap`). It rejects every swap it did not start, requires fee 0 and full-range liquidity so the pool is exactly $xy = k$, and collects orders into 5-block batches. A permissionless `settle()` runs the two-sided auction and makes one residual swap inside v4's `unlock`.
- **ClinchEngine**, one Solidity library used by both. It matches a Python reference engine bit for bit over 768 fuzzed runs, and a TypeScript port in the app replays each settled auction step by step.
- **A World ID arm.** The same launch, but paid in a voucher token that only a World ID verified person can mint, capped per person, with one wallet per person. It shows that per-person limits can sit on top of Clinch without changing the core contracts.

Nothing has an owner or an upgrade path. A keeper script can settle automatically, but anyone can settle from any wallet or a block explorer.

## 5. Results

**Incentives.** We searched for profitable deviations with random search plus hill climbing, 200 instances × 400 tries per mode. The largest gain found, as a fraction of the deviator's budget:

| Deviation | $\varepsilon = 1\%$ | $\varepsilon = 0.1\%$ | Cases above $\varepsilon$ |
|---|---|---|---|
| Pool, split into up to 8 wallets | $2.8 \times 10^{-3}$ | $1.5 \times 10^{-5}$ | 0 |
| Pool, misreported value | $2.5 \times 10^{-3}$ | $9.5 \times 10^{-6}$ | 0 |
| Launch, split | $2.8 \times 10^{-5}$ | $8.5 \times 10^{-8}$ | 0 |
| Two-sided, wallets on the other side | $1.7 \times 10^{-3}$ | $4.2 \times 10^{-6}$ | 0 |
| Risk-free profit bot | 0 | 0 | 0 |

Gains stay well under $\varepsilon$ and shrink faster than $\varepsilon$ does. This is a search, not a proof: on a discrete ladder the guarantee is only approximate, and we have not proven the bound.

**Against other designs.** In the whale example, Clinch is the only design where the truthful report was the best one found, and splitting lowered utility (680 with one wallet, 270 with 100) instead of raising it. Note what this does not mean: Clinch does not limit whales. The truthful whale won 87% of supply, versus 71% under uniform price. The guarantee is about incentives, not distribution.

**Sandwiches.** On a plain 10,000/10,000 pool a sandwich bot earned 0.5 to 55 units per victim. In a Clinch batch it earns nothing, because everyone in the batch faces the same clock and matched volume does not move the pool.

**Tests and gas.** 99 contract tests pass, with 2,000 fuzz runs per property. Settling costs about 1,200 to 1,800 gas per bid per ladder step: 1.38M for a 32-order batch. We use a 0.1% step on pools and 1 to 1.5% plus a price ceiling for launches.

**On Sepolia,** against Uniswap's official PoolManager, with every contract verified on Etherscan:

- A 1,000,000-token launch with six bidders settled at $0.42 in 99 steps (3.68M gas), raised 252,631.85 USDC, and seeded a CLNCH/USDC pool. The in-browser replay matches the chain exactly.
- A WETH/USDC batch of 1 WETH against 2,300 USDC of buys cleared at $3,048.73 to $3,048.80 against a spot of $3,048.92, for 1.02M gas. A wallet with no orders settled it and claimed for everyone.
- A direct swap through Uniswap's own test router reverted with `DirectSwapDisabled`.

## 6. Limitations

- Incentive compatibility is approximate on the discrete ladder; we measured the error but have not proven a bound.
- We assume block builders cannot censor chosen bids.
- Each unit is paid at the price where it was clinched, so bidders can pay different average prices.
- Budgets are locked for the whole sale or batch.
- Very large contested launches do not fit in one block. A multi-transaction ladder would fix this; we have not built it.
- Pool LPs earn no swap fees, since hook pools are fee 0.
- The code is not audited.

---

**References**


[1] C. Zhang, Y. Li, E. Shi, M. Zhang, L. Xia, Q. Han. Sybil-proofness of generalized adaptive clinching and implications for blockchain mechanism design. arXiv:2610.36629, 2026.

[2] S. Dobzinski, R. Lavi, N. Nisan. Multi-unit auctions with budget limits. FOCS 2008; *Games and Economic Behavior* 74(2), 2012.

[3] L. M. Ausubel. An efficient ascending-bid auction for multiple objects. *American Economic Review* 94(5), 2004.

[4] S. Bhattacharya, V. Conitzer, K. Munagala, L. Xia. Incentive compatible budget elicitation in multi-unit auctions. SODA 2010.

[5] E. Shi, M. Zhang, H. Chung, Y. Li. Otter: a provably MEV-resilient AMM via surplus redistribution. arXiv:2609.03474, 2026.

[6] H. Adams et al. Uniswap v4 Core. Whitepaper, 2024.

[7] Uniswap Labs. Continuous Clearing Auction. docs.uniswap.org, 2025.

[8] P. Daian et al. Flash Boys 2.0: frontrunning in decentralized exchanges, miner extractable value, and consensus instability. IEEE S&P 2020.

[9] E. Budish, P. Cramton, J. Shim. The high-frequency trading arms race: frequent batch auctions as a market design response. *QJE* 130(4), 2015.